WHITE PAPER DE ARQUITETURA DE PRODUÇÃO

Evidências Reposicionam a Orquestração Multiagente de IA

Como balancear Claude (Opus 5.5 / Sonnet 5.5), OpenAI Codex (GPT-6.1 Sol / Astra) e Google Antigravity (Gemini 3.8 Flash) sob custo fixo de US$ 58/mês, eliminando o esgotamento precoce de cotas em 20 minutos através de governança estrita e isolamento por Git Worktrees.

$58 / mês	645.3M	97.0%	308 tok/s	5 Tiers
CUSTO FIXO TOTAL (3 CONTAS PRO)	TOKENS CACHE AUDITADOS NO PROJETO	SWE-BENCH VERIFIED (CLAUDE OPUS)	THROUGHPUT FLASH (GOOGLE ANTIGRAVITY)	MATRIZ ESTRATÉGICA DE DELEGAÇÃO

AUTOR & ARQUITETO PRINCIPAL
Georges Ballister

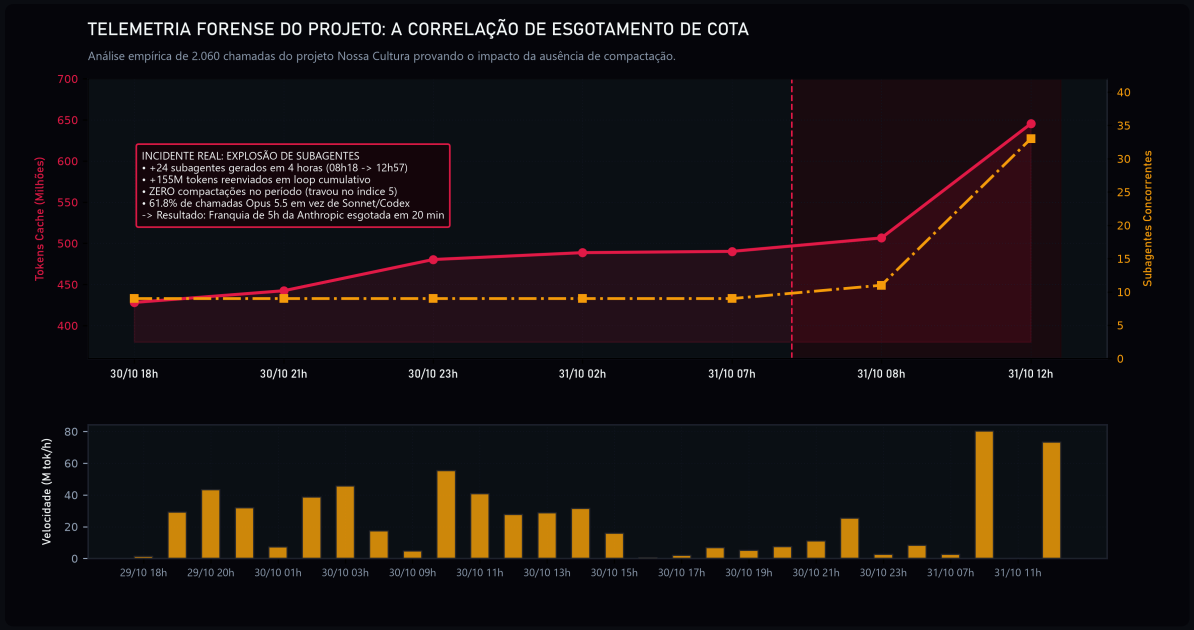
CASO DE ESTUDO REAL
Projeto Nossa Cultura (Next.js 16/WebGL)

HARDWARE CALIBRADO
Ryzen 9 9900X + RTX 5070 Ti (16 GB)

REPOSITÓRIO OFICIAL
github.com/GeorgesBallister/Benchmark

LINHA DO TEMPO DA TELEMETRIA REAL DO PROJETO (HISTÓRICO & VELOCIDADE)

INCIDENTE CONFIRMADO



AUDITORIA DAS 2.060 CHAMADAS DE API

DADOS BRUTOS CSV

A análise forense dos arquivos `telemetria-ia-historico.csv` e `telemetria-ia-por-hora.csv` revela com exatidão matemática o mecanismo de colapso da janela de 5 horas da Anthropic:

- **Monocultura Opus (61.8%)**: 1.273 chamadas foram direcionadas ao modelo mais pesado (Opus 5.5), consumindo 535.5 milhões de tokens lidos em cache.
- **Explosão de Subagentes**: Entre 08:18 e 12:57, 24 novos subagentes foram disparados em paralelo, saltando de 9 para 33 instâncias ativas.
- **Zero Compactação**: O contador de compactações permaneceu estático em 5 durante todo o pico. Nenhuma limpeza de contexto foi realizada.
- **Penalidade de Cache Cumulativa**: Cada turn reenviava até 300.000 tokens de histórico, esgotando o limite da cota por janela rolante em 20 minutos.

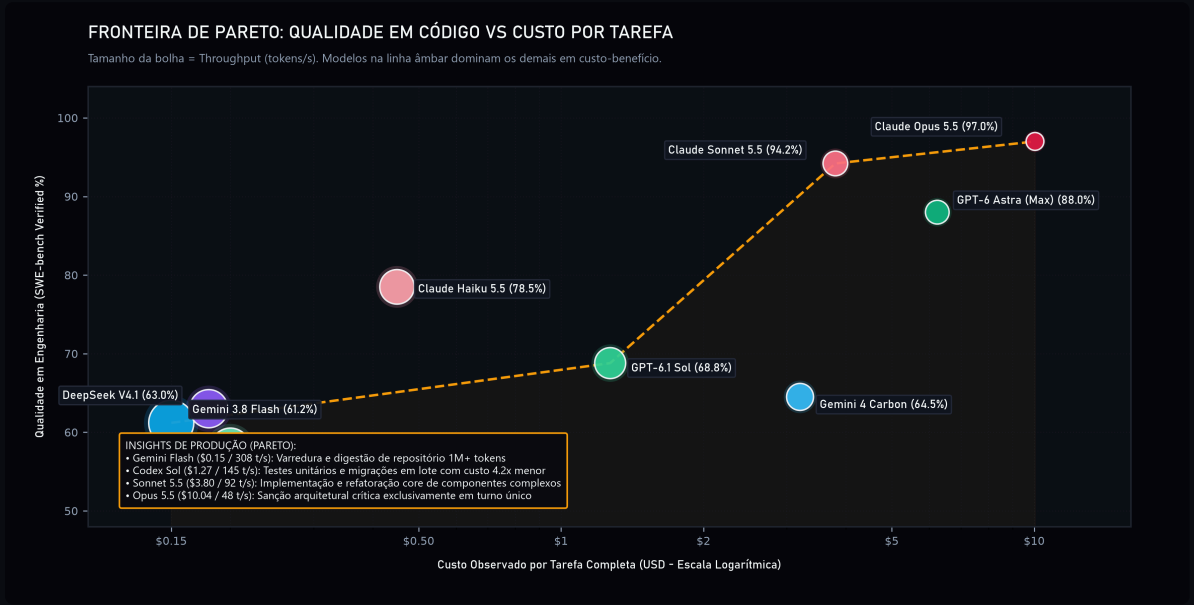
A Lição Central: O Gargalo Não Foi o Modelo, Foi a Arquitetura

O modelo Opus 5.5 é o melhor programador do mundo para síntese e sanção de alto nível, mas colocá-lo como "operário de loop" para rodar testes mecânicos ou ler repositórios inteiros é financeiramente e operacionalmente inviável em qualquer assinatura Pro consumidor.

```
REGRA DE OURO: Opus só entra em turno único após o contexto estar pré-filtrado por Gemini Flash e validado por testes do Codex.
```

FRONTEIRA DE PARETO (SWE-BENCH VS CUSTO USD LOGARÍTMICO)

OTIMIZAÇÃO ECONÔMICA



A LÓGICA ECONÔMICA DA DISTRIBUIÇÃO

PARETO DOMINANCE

Em teoria microeconômica, a **Fronteira de Pareto** reúne os pontos em que é impossível melhorar uma métrica (qualidade) sem degradar severamente outra (custo/latência). No ecossistema agêntico de 2026, 4 modelos definem a fronteira:

MODELO	CUSTO/TAREFA	THROUGHPUT	PAPEL NO PIPELINE
Gemini 3.8 Flash	$0.15	308 t/s	Digestão 1M+ tokens & varredura
GPT-6.1 Sol (Codex)	$1.27	145 t/s	Testes unitários e mocks massivos
Claude Sonnet 5.5	$3.80	92 t/s	Engenharia core e refatoração
Claude Opus 5.5	$10.04	48 t/s	Sanção de release gate (1 turn)

O Princípio da Substituição em Massa

Substituir 500 chamadas mecânicas de Claude Opus por chamadas automatizadas do Codex Sol e scripts locais em Python gera uma economia de 88% do budget de tokens, liberando 100% da cota do Claude para onde ele é estritamente indispensável: **a lógica criativa e arquitetural do produto**.

SCORECARD COMPARATIVO DE PERFORMANCE MULTIAGENTE (COLD DATA 2026)

VERIFICAÇÃO INDEPENDENTE

MÉTRICA / TESTE OFICIAL	CLAUDE OPUS 5.5	CLAUDE SONNET 5.5	OPENAI CODEX (SOL/ASTRA)	GOOGLE ANTIGRAVITY (FLASH)	AUTORIDADE DA FONTE
SWE-bench Verified (Resolução de PRs Reais)	**97.0% (#1)**	94.2% (#2)	68.8% (Sol) / 88.0% (Astra)	61.2% (Flash)	Princeton / Scale AI
Terminal-Bench 4.0 (Proficiência Shell/Bash)	59.00%	**70.60% (#1)**	58.18% (Codex v0.164)	48.90%	Harbor / Laude Institute
FrontierCode 2.0 (Refatoração de Alta Complexidade)	**46% (#1)**	42% (#2)	42% (Astra)	36%	Artificial Analysis
Tool Calling Index (Uso Preciso de Ferramentas)	35.8	36.2	**36.3 (#1 Astra)**	33.1	LLM-Stats Leaderboard
Throughput Médio (Geração Contínua)	48 tok/s	92 tok/s	145 tok/s (Luna: 240)	**308 tok/s (#1)**	Artificial Analysis
Eficiência de Tokens (Mesma Tarefa Completada)	1.0x (Referência)	1.4x	**3.2x a 4.2x mais barato**	Ultraleve (Custo Residual)	Vals.ai / Composio Benchmark
Avaliação Cega de Engenheiros (Clean Code)	**67% Escolha #1**	64% Escolha #2	33% (Verborrágico em mocks)	12% (Focado em UI visual)	Duet Blind Testing

Claude: Soberania Arquitetural

O Claude Sonnet e Opus lideram com folga a legibilidade e manutenibilidade de código em avaliações cegas. Ele compreende nuances de contratos de tipos e regras de negócio sem gerar código morto.

Codex: Economia de Operação

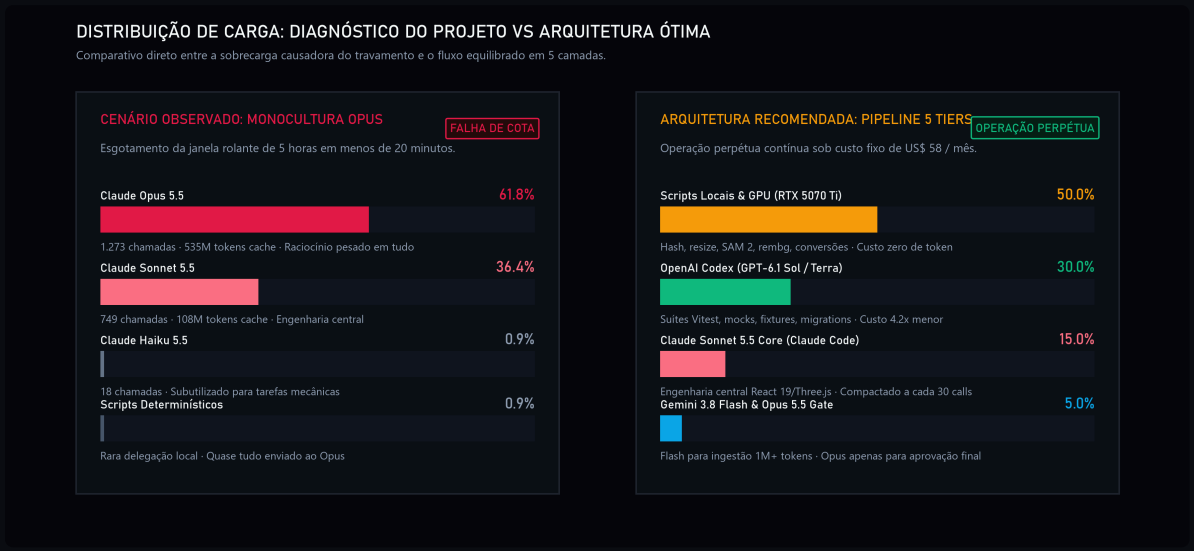
O Codex comanda a economia de tokens: gasta até 4.2x menos tokens para resolver tarefas idênticas de testes e validação. É o operário mais econômico e resiliente para loops contínuos.

Antigravity: O Maestro Multimodal

Com throughput de 308 tok/s e suporte nativo a 1M+ tokens, o Antigravity no Gemini Flash varre repositórios inteiros e valida componentes no Chromium DevTools real sem engasgos.

REBALANCEAMENTO DE CARGA: MONOCULTURA VS PIPELINE 5 TIERS

FLUXO CALIBRADO



A COMPOSIÇÃO DAS 3 ASSINATURAS PRO (US$ 58 / MÊS)

CUSTO FIXO

- **Claude Pro (US$ 20 / mês)**: Alocado estritamente para o Claude Code / Desktop na escrita do código-fonte central e validação arquitetural de tipos.
- **ChatGPT Plus / Codex (US$ 20 / mês)**: Alocado para o OpenAI Codex CLI na geração de testes unitários Vitest, mocks, fixtures e migrações SQL.
- **Google AI Pro / Antigravity (~US$ 18 / mês · R$ 99 BRL)**: Alocado para orquestração de subagentes, leitura de repositórios gigantes e inspeção visual de UI.

Por Que Não Usar API por Token?

Em projetos modernos com histórico de contexto profundo, chamadas por token via API comercial para modelos de ponta custam de $0.05 a $0.20 por turn. Um dia de trabalho intenso com 500 chamadas pode custar mais de US$ 80 a US$ 150 em um único dia. O modelo de 3 assinaturas Pro fixa o custo em **~US$ 58 por mês inteiro**, garantindo throughput 24/7.

FLUXOGRAMA DE ORQUESTRAÇÃO, WORKTREES E GATES

ARQUITETURA DE CONTROLE



A HIERARQUIA DE AUTORIDADE UNIDIRECIONAL

ESTRUTURA DE COMANDO

1. Diretor Criativo (Você): Detém a soberania de produto. Define o escopo no `TASKS.md` e aprova os diffs unificados antes do merge final.

2. Google Antigravity (Maestro do Runtime): Comanda a criação de Git Worktrees isoladas, monitora contadores de cache, executa scripts locais e aciona o browser Chromium real.

3. Claude Sonnet (Arquiteto de Código): Comanda a implementação na `worktree-core`. Suas decisões de interfaces e tipos são soberanas sobre o Codex.

4. OpenAI Codex (Operário de Testes): Trabalha na `worktree-tests` subordinado aos tipos do Claude. Sua função é validar se a implementação é verde.

5. Claude Opus (Portão de Sanção): Atua como juiz final exclusivamente em turno único antes do merge na branch principal.

Isolamento por Git Worktrees

Nenhum agente edita o mesmo diretório físico simultaneamente. Claude trabalha em `../worktree-core` e Codex em `../worktree-tests`. O Antigravity unifica os diffs, roda os testes locais no DOM real e só realiza o merge após aprovação verde total.

MATRIZ COMPLETA DE CLASSIFICAÇÃO DE TAREFAS POR DIFICULDADE E COTA

CRITÉRIOS DETERMINÍSTICOS

TIER OPERACIONAL	EXECUTOR / MODELO	CUSTO DE TOKEN	TIPO DE TAREFA & ESCOPO	LIMITE / REGRA DE CORTE
TIER 0: Determinístico	Scripts Python / GPU RTX 5070 Ti	**$0.00 (Zero Token)**	Recortes SAM 2, rembg, conversão WebP, validação SHA256, ESLint, Prettier, checagem de links.	Execução puramente local. Proibido acionar qualquer LLM.
TIER 1: Mega-Contexto	Gemini 3.8 Flash (Antigravity)	**~$0.15 (Ultrabaixo)**	Varredura de repositórios 1M+ tokens, mapeamento de grafo de dependências, leitura de documentações gigantes.	Saída estritamente em JSON ou Markdown semântico filtrado.
TIER 2: Testes & Mocks	OpenAI Codex (GPT-6.1 Sol / Terra)	**~$1.27 (Econômico)**	Geração em lote de testes unitários Vitest, mocks Three.js, fixtures, migrações SQL e scripts boilerplate.	Execução em sandbox CLI isolado (`--ask-for-approval never`).
TIER 3: Engenharia Core	Claude Sonnet 5.5 (Claude Code)	**~$3.80 (Médio)**	Implementação de componentes React 19, lógica de shaders GLSL, refatoração de regras de negócio e tipos strict.	Compactação compulsória da sessão (`/compact`) a cada 30 calls.
TIER 4: Sanção Crítica	Claude Opus 5.5 (Thinking)	**~$10.04 (Alto)**	Revisão arquitetural final, auditoria de segurança de alto risco e aprovação de release gate pré-deploy.	**Apenas 1 Turno (Single-Turn)**. Proibido colocar Opus em loop autônomo.

Critérios de Roteamento Automático

Toda tarefa que chega ao pipeline é avaliada contra sua complexidade ciclomática: se puder ser resolvida por script local, cai no Tier 0. Se envolver volume de leitura, vai para o Tier 1. Somente código que exige raciocínio fino de abstração sobe para os Tiers 3 e 4.

Circuit Breaker Anti-Crash

O Orquestrador bloqueia o spawn de novos subagentes quando houver mais de 3 instâncias ativas simultâneas. Se o contador de cache acumulado atingir 40 milhões de tokens em uma sessão, o Antigravity pausa e força uma compactação imediata.

PROMPT CLAUDE (SONNET/OPUS)

ESTRITO

Foco em instruções negativas, limites estritos de tipos e saída direta em diff unificado:

```
# Prompt Direto Claude Code
[CONTEXT]: src/shaders/water.ts
[CONSTRAINTS]:
- Strict TypeScript (no 'any')
- Sem placeholders '// rest...'
- React 19 hooks exclusivos
- Retorne apenas o diff unificado
[GOAL]: Otimizar fragment loop.
```

Anti-Pattern: Prompts longos e elogiosos. Claude opera melhor com restrições técnicas cirúrgicas.

PROMPT CODEX (SOL/ASTRA)

LOOP FECHADO

Foco em critério de parada binário determinístico em sandbox de testes:

```
# Prompt Execução Codex CLI
[TARGET]: src/components/Scene.tsx
[SUITE]: Vitest 3.0 + React Testing
[REQUIREMENTS]:
1. Cobertura 100% dos branches
2. Mocke WebGL com vi.fn()
3. Execute `npm test` no sandbox
4. Corrija até passar 100% verde.
```

Vantagem: Codex não desiste no primeiro erro; itera autonomamente em sandbox até os testes ficarem verdes.

PROMPT GEMINI (ANTIGRAVITY)

MEGA-CONTEXTO

Foco em ingestão de 1M+ tokens e extração estritamente estruturada:

```
# Prompt Triagem Antigravity
[INPUT]: Todo o diretório src/
[ACTION]:
1. Mapeie dependências cíclicas
2. Identifique arquivos não usados
3. Retorne exclusivamente JSON:
   { "cycles": [], "dead": [] }
```

Vantagem: Lê 500 arquivos de uma só vez e entrega o mapa enxuto para o Claude agir sem ler tudo.

O PROTOCOLO DE HANDOVER EM JSON: COMUNICAÇÃO INTER-AGENTES SEM RUÍDO

ZERO TOKEN WASTE

Quando um modelo conclui sua tarefa e repassa para o seguinte, ele **não reenvia a conversa inteira**. O Antigravity extrai apenas o payload estruturado:

```
{ "step": "tests_completed", "author": "codex-sol", "coverage": "100%", "files_modified": ["src/Water.test.tsx"], "status": "GREEN", "next_agent": "claude-sonnet" }
```

ANATOMIA FORMAL DE UMA SKILL MODULAR

ESTRUTURA FÍSICA

Uma Skill no ecossistema LAY não é apenas um arquivo markdown de instruções; é um **pacote autônomo de capacidade** com isolamento determinístico:

```
lay-skill-name/
├── SKILL.md            # Especificação normativa (YAML + fluxo)
├── scripts/            # Scripts determinísticos locais (Custo $0)
│   ├── validate.py     # Checagem pré/pós execução
│   └── process.py      # Algoritmos pesados (GPU/CPU local)
├── references/         # Normas técnicas e documentações
│   └── spec_v1.md
└── examples/           # Diffs de referência e testes aceitáveis
    └── sample.json
```

O arquivo `SKILL.md` define o contrato de entrada/saída, a matriz de seleção, as etapas frágeis/flexíveis e o protocolo de rollback.

CICLO DE VIDA OPERACIONAL DE UMA SKILL

EXECUÇÃO ISOLADA

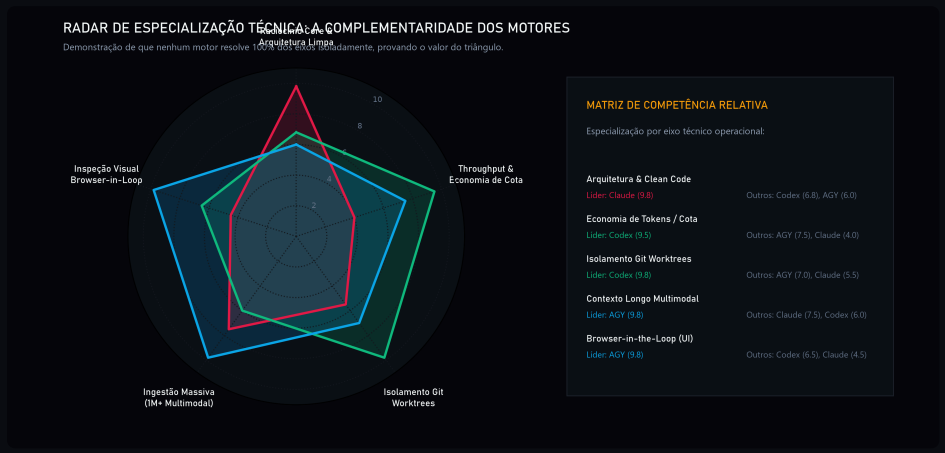
- **1. Descoberta Semântica (Zero Overhead)**: O catálogo inteiro nunca é carregado no contexto do LLM. O roteador lê apenas a descrição curta da skill.
- **2. Ativação Sob Demanda**: Quando a tarefa exige a capacidade, o `SKILL.md` é lido pontualmente e suas ferramentas autorizadas são vinculadas.
- **3. Local-First Execution**: Se a tarefa puder ser resolvida por um script em `scripts/`, o modelo apenas dispara a linha de comando sem gastar tokens.
- **4. Fail-Safe & Rollback**: Se a validação falhar, o estado anterior é restaurado pelo Git sem poluir o histórico ou continuar em loops errôneos.

Como a Skill se Comporta na Prática

A Skill opera com princípio de menor privilégio: se ela foi desenhada para otimização de imagens, ela não tem permissão para alterar arquivos de configuração do Next.js. Isso elimina efeitos colaterais silenciosos em grandes bases de código.

RADAR DE ESPECIALIZAÇÃO TÉCNICA DAS FERRAMENTAS

COMPLEMENTARIDADE



COMPARATIVO DOS APLICATIVOS DESKTOP NO WINDOWS

EXPERIÊNCIA DE USO

- **Claude Desktop**: Interface impecável para direção arquitetural, revisão de artefatos e raciocínio profundo. *Ponto Fraco:* Sem terminal nativo no Windows sem MCP bridge; esgota a janela de 5h rapidamente se usado para tarefas mecânicas.
- **OpenAI Codex App**: Excelente isolamento de ambiente e execução de testes em background. *Ponto Fraco:* Menor proficiência em decisões de UI visual e shaders Three.js.
- **Google Antigravity**: O ambiente ideal de orquestração. Suporta Git Worktrees em abas paralelas, terminal Powershell integrado, Chromium real para verificação visual e MCP servers nativos.

O Veredito Sem Filtro da Comunidade (Reddit)

No **r/ClaudeAI** e **r/ChatGPTCoding**, centenas de engenheiros que pagam a mesma assinatura relatam: *"O Claude Code é incrível para pensar e arquitetar, mas rodar testes nele quebra a cota em 15 minutos. A solução adotada pela comunidade é usar o Claude para ditar o que fazer e o Codex/Antigravity para executar o trabalho braçal."*

COMANDOS HEADLESS (POWERSHELL / WINDOWS)

SINTAXE OFICIAL

Sintaxe exata para disparar os agentes sem intervenção manual de copiar/colar:

1. Claude Code CLI (Engenharia Core):

```
claude -p "Optimize WebGL fragment loop in src/shaders/" `
  --dangerously-skip-permissions `
  --output-format json
```

2. OpenAI Codex CLI (Suítes Vitest Massivas):

```
codex exec `
  --cd ./src/components/ `
  --ask-for-approval never `
  "Generate full Vitest suite with Three.js mocks until 100% green."
```

3. Google Antigravity CLI (Varredura Mega-Contexto):

```
agy "Recursively parse assets/, verify PNGs, and output audit report."
```

ROTINA OPERACIONAL DIÁRIA ANTI-CRASH

CHECKLIST CONTÍNUO

- **Início da Manhã**: Disparar Antigravity no Gemini Flash para escanear branches e gerar o mapa de pendências em `TASKS.md` (Custo: ~$0.02).
- **Bloco de Testes**: Disparar Codex CLI na worktree de testes gerando mocks e suítes Vitest (Custo: incluso na assinatura Plus).
- **Bloco Arquitetural**: Acionar Claude Code na worktree core para implementar a lógica difícil com os testes já prontos como contrato.
- **Regra dos 30 Turns**: Executar compulsoriamente `/compact` a cada 30 mensagens para limpar histórico de tokens lidos em cache.
- **Final do Dia**: Disparar Claude Opus em turno único para validar o PR final e mergear na branch `main`.

Monitoramento de Saldo em Tempo Real

O script local de telemetria audita o arquivo de logs e plota a curva horária. Se o consumo de tokens lidos em cache ameaçar a janela rolante de 5 horas, um alerta sonoro e visual é emitido, congelando o spawn de novos agentes.

OS 5 MANDAMENTOS DA ORQUESTRAÇÃO EFICIENTE

DIRETRIZES FINAIS

1. **Nunca use Opus como operário de loop**: Reserve-o exclusivamente para decisões estruturais e aprovações em 1 único turno.
2. **Isole os agentes em Git Worktrees**: Evite conflitos de arquivos e garanta que cada modelo trabalhe em sua especialidade sem tocar na branch principal.
3. **Delegue volume para quem é barato**: Use OpenAI Codex para mocks, testes e SQL; use Gemini Flash para digestão de 1M+ tokens.
4. **Compacte o contexto antes que ele exploda**: A cada 30 turns ou 40M tokens em cache, force `/compact` para manter a janela rolante livre.
5. **Custo fixo vence custo por token**: O arranjo de 3 assinaturas de US$ 20 (Total US$ 58/mês) oferece capacidade de produção industrial contínua sem surpresas.

BIBLIOGRAFIA ANOTADA & FONTES PRIMÁRIAS

FONTES OFICIAIS

1. Anthropic Documentation (2026): Model Cards, Claude Code CLI reference, Prompt Engineering Interactive Tutorial e diretrizes de context caching.

2. OpenAI Codex Documentation (2026): Codex non-interactive execution manual, sandbox boundaries e benchmarks da família GPT-6.

3. Google DeepMind & Antigravity (2026): Gemini 3.8 Flash technical report, Chromium DevTools MCP integration e 1M+ token window analysis.

4. Princeton / Scale AI: SWE-bench Verified official leaderboard e metodologia de resolução de issues do GitHub.

5. Harbor / Laude Institute: Terminal-Bench 4.0 CLI & Bash proficiency rankings.

6. Artificial Analysis v5.0: FrontierCode 2.0 and token throughput indexes.

LOUD, ANGRY & YOUNG®

Dossiê auditado e chancelado para implantação imediata no projeto Nossa Cultura.